

Training Report

on

Algorithmic Trading in HFT

Submitted in partial fulfillment of the requirements

For the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

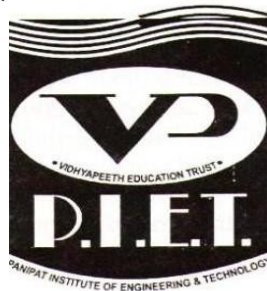
(ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)

Submitted By

Name - Mahak Rathee

Roll No. - 2823827(2023057074)

(Batch- 2023-2027)



Panipat Institute of Engineering & Technology,

(Affiliated to Kurukshetra University Kurukshetra, India)

Samalkha, Panipat

(Session 2025-26)

CANDIDATE’S DECLARATION

I hereby declare that the work presented in this training report entitled “**Algorithmic Trading in HFT**”, submitted in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in **Btech CSE(AIDS)**, submitted to **Panipat Institute of Engineering & Technology, Kurukshetra University, Kurukshetra, India** is an authentic record of my own work carried out during the period from **10/07/2025** to **14/08/2025** under the guidance of **Sagar(Sr. Manager, WeSecure Technologies)**.

The work reported in this project report has not been submitted by me for the award of any other degree or diploma.

Date: 12/09/2025

Student’s name- Mahak Rathee

Place: WeSecure Technologies,Pritampura,Delhi,110034

Roll No.- **2823827**

CERTIFICATE



WESECURE TECHNOLOGIES
GSTIN - 07AAEFW2839D1Z7

INTERNSHIP CERTIFICATE

Ref. No: WS/005/Internship/2025-26

Date: 29 Aug 2025

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Miss. Mahak Rathee** (Roll Number: **2823827**), student of **B. Tech in Artificial Intelligence and Data Science (AI & DS)**, Panipat Institute of Technology, has successfully completed her internship at Wesecure Technologies, New Delhi, from 10th July 2025 to 20th August 2025.

During this period, she worked as an Intern in the High-Frequency Trading (HFT) Project under the Information Technology & Research Division. She was provided exposure to:

- Algorithmic Trading and HFT systems.
- Data preprocessing and analysis of financial tick data.
- Machine Learning models for signal generation and trade prediction.
- Back testing and performance evaluation of trading strategies.
- Integration of analytical dashboards for live and historical trade monitoring.

Her performance during the internship was found to be sincere, dedicated, and enthusiastic. We hereby acknowledge her successful completion of the internship and wish her the best for her future academic and professional pursuits.



Yours sincerely,

 29/08/2025

Narender Singh

Chief Technology Officer

Registered Address : 391, Third floor, Block-C, Sector-19, Divya Jyoti Apartments, Rohini, New Delhi-110089

Corporate Address : RU-394, Ground floor, Near Pillar No-265, outer Ring Road Pitampura, New Delhi -110034

 011-45567102  sales@wesecuretech.in  www.wesecuretech.in

Acknowledgement

I would like to express my heartfelt gratitude to my college faculty and project guide, **Sagar**, for their unwavering support and guidance throughout the duration of this project. Their expertise, encouragement, and mentorship have played a crucial role in shaping the project's direction and ensuring its successful completion. I am also thankful to my fellow classmates who have actively contributed to the project with their valuable insights and collaborative effort.

I would like to acknowledge the college administration for providing us with the necessary resources and facilities to carry out this project. Their constant support and belief in our abilities have been instrumental in our project's accomplishment. Additionally, I extend my appreciation to my family and friends for their continuous encouragement and understanding during this challenging yet rewarding journey.

Name- Mahak Rathee

ABSTRACT

This project focuses on the design and implementation of an **algorithmic trading system in the context of High-Frequency Trading (HFT)**. The objective is to develop a strategy capable of analyzing tick-level data from the NSE (National Stock Exchange) and executing trades automatically with minimal latency. The system integrates traditional technical indicators such as RSI and SMA with machine learning models to generate buy and sell signals. Additionally, advanced risk management techniques including ATR-based stop-loss and take-profit mechanisms were incorporated to enhance profitability while controlling drawdowns.

A backtesting module was developed to evaluate strategy performance under realistic trading conditions, accounting for brokerage charges and slippage. Key metrics such as win rate, Sharpe ratio, drawdown, and overall profitability were computed to measure effectiveness. To ensure usability, an interactive **Streamlit dashboard** was created, providing real-time monitoring, trade visualization, and decision support. The project also explores the integration of Power BI for advanced reporting and performance analysis.

The results demonstrate that combining statistical indicators with machine learning filters can improve trade selection and increase profitability in high-frequency environments. This work highlights the potential of algorithmic trading systems in HFT, emphasizing the importance of speed, data-driven decision-making, and robust evaluation in modern financial markets.

- Designed an **algorithmic trading system** for High-Frequency Trading (HFT) using NSE tick data.
- Implemented strategies with **RSI, SMA, and machine learning models** for trade signal generation.
- Added **ATR-based stop-loss and take-profit** for effective risk management.
- Built a **backtesting and live trading dashboard** using Streamlit with Power BI integration.
- Achieved improved **profitability, win rate, and risk-adjusted performance** through data-driven decision making.

CONTENTS (16 PTS.)

Candidate's Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
List of Figures	v
List of Tables	vi
List of Graphs	vii

Chapter No	Description	Page No.
Chapter 1:	INTRODUCTION	1-2
Chapter 2:	Software and hardware requirements	3-5
Chapter 3:	Software requirement analysis	6-10
Chapter 4:	Coding	11-32
Chapter 5:	Output Screens	33-35
Chapter 6:	Conclusion	36
Chapter 7:	Further enhancement/recommendations	37
Chapter 8:	References and Bibliography	38

List of Figures

Figure no.	Figure Name	Page no.
Figure 3.1	Given data	7
Figure 5.1	Home page of jupyter notebook	25
Figure 5.2	First page of project	26

List of Tables

Table no.	Table Name	Page no
Table 4.1	User Title return data	26

List of Graphs

Graph no.	Graph Name	Page no
Graph 4.1	Age return	18
Graph 4.2	User Title and Age	19
Graph 4.3	User Title and Return	20
Graph 4.4	Return	20
Graph 4.5	Heatmap	20
Graph 4.6	Delivery duration	23
Graph 4.7	Brand Id	24

Graph 5.1	Top 10 item size returned	26
Graph 5.2	Top 10 brand id returned	27

Chapter-01

INTRODUCTION

In recent years, financial markets have undergone a profound transformation with the rise of algorithmic trading (algo trading) and high-frequency trading (HFT). Traditional manual trading methods, which relied on intuition and delayed reactions, have been replaced by automated systems capable of executing thousands of trades within fractions of a second. The essence of algorithmic trading lies in using mathematical models, technical indicators, and machine learning algorithms to identify profitable opportunities and place orders with minimal human intervention. In this context, High-Frequency Trading, a specialized form of algorithmic trading, emphasizes speed, efficiency, and precision by leveraging low-latency data feeds and rapid execution mechanisms.

This project focuses on building a comprehensive algorithmic trading framework that utilizes tick-level data from the National Stock Exchange (NSE) to design, test, and execute trading strategies. Unlike daily or hourly data, tick data provides granular insights into price movements and market microstructure, making it highly suitable for HFT applications. The core objective of this project is to create a system that can generate reliable buy and sell signals, execute trades in real time, and manage risks effectively to maximize profitability.

To achieve this, the system integrates traditional technical indicators such as the Relative Strength Index (RSI) and Simple Moving Average (SMA) with machine learning models for enhanced signal prediction. While indicators capture momentum and trend-following opportunities, the machine learning component filters out noise and improves trade selection. In addition, robust risk management techniques have been implemented, including the Average True Range (ATR)-based stop-loss and take-profit mechanism, ensuring that trades are exited at optimal points to balance risk and reward.

A critical part of the project is the development of a backtesting module. Backtesting allows strategies to be validated on historical data under realistic conditions, taking into account brokerage charges, transaction costs, and slippage. Performance evaluation metrics such as win rate, Sharpe ratio, maximum drawdown, and overall profitability are used to assess the robustness and effectiveness of the strategy. This ensures that the system is not only theoretically sound but also practically viable in real trading environments.

For visualization and usability, the project incorporates a Streamlit-based interactive dashboard. The dashboard provides users with an intuitive interface to switch between live trading and backtesting modes, monitor trade execution, visualize performance curves, and adjust strategy parameters dynamically. To complement this, Power BI dashboards were also designed for advanced reporting and deeper insights into performance metrics, enabling clear presentation of results during analysis or stakeholder review.

The project demonstrates that the fusion of statistical techniques, technical indicators, and machine learning models can significantly enhance the decision-making process in HFT. It highlights the importance of automation, speed, and data-driven approaches in modern financial markets. By building an end-to-end system—covering strategy design, backtesting, risk management, live monitoring, and reporting—this work contributes to the growing field of quantitative finance and showcases the potential of algorithmic trading in delivering consistent and scalable performance.

Ultimately, the project reflects the evolution of trading practices in the digital era, where technology, algorithms, and analytics converge to redefine market participation. It provides a solid foundation for future exploration of advanced topics such as reinforcement learning, deep learning models for trade prediction, and optimization of execution strategies, thereby opening pathways to more sophisticated applications in the world of HFT.

Chapter-02

SOFTWARE AND HARDWARE REQUIREMENTS

1) Jupyter Notebook- It is a server-client application that allows editing and running notebook documents via a web browser. It can be executed on a local desktop requiring no internet access (as described in this document) or can be installed on a remote server and accessed through the internet.

In addition to displaying/editing/running notebook documents. It has a "Dashboard" (Notebook Dashboard), a "control panel" showing local files and allowing to open notebook documents or shutting down their kernels. Jupyter notebooks are especially useful for "showing the work" that your data team has done through a combination of code, markdown, links, and images. They are easy to use and can be run cell by cell to better understand what the code does. Jupyter notebooks can also be converted to a number of standard output formats (HTML, PowerPoint, LaTeX, PDF, Restructured Text, Markdown, Python) through the web interface. This flexibility makes it easy for data scientists to share their work with others.

Jupyter notebooks can also be converted to a number of standard output formats (HTML, PowerPoint, LaTeX, PDF, Restructured Text, Markdown, Python) through the web interface. This flexibility makes it easy for data scientists to share their work with others.

Jupyter Notebook was used as the primary environment for developing, testing, and experimenting with the algorithmic trading strategies. It provided an interactive platform to write Python code, integrate libraries such as Pandas, NumPy, scikit-learn, and TA indicators, and perform exploratory data analysis on tick-level NSE data. Jupyter enabled quick data visualization using Matplotlib, helping in understanding market trends, trade signals, and profit curves during development. It also supported step-by-step testing of strategies, machine learning models, and risk management logic. Overall, Jupyter served as the research and prototyping tool before deploying the system into dashboards.

2)PowerBI- Power BI played a significant role in this project by providing a platform for data visualization, reporting, and advanced analysis of the trading strategies developed for High-Frequency Trading (HFT). While the algorithmic trading system and Streamlit dashboard were primarily designed for live execution and interactive monitoring, Power BI was integrated to enhance post-trade analysis and performance reporting.

The key advantage of using Power BI lies in its ability to handle large volumes of financial data efficiently and present them through interactive dashboards and visualizations. The tick-level data from the National Stock Exchange (NSE), along with backtest results, were imported into Power BI for detailed exploration. This enabled clear representation of important metrics such as total profit, win rate, Sharpe ratio, drawdown levels, and trade frequency. Charts like line graphs, bar charts, and histograms were used to track profit curves, trade distribution, and risk exposure.

Power BI's drill-down and filter capabilities provided flexibility to analyze results across different time frames, instruments, and strategy parameters. For example, users could filter performance based on specific dates or compare strategy outcomes under different market conditions. Such insights allowed a deeper understanding of how strategies behave in volatile versus stable markets.

Additionally, Power BI dashboards served as an effective tool for project documentation and presentation. Instead of static reports, stakeholders were presented with interactive reports where they could explore results in real time. This added significant value in communicating the project's outcomes clearly and professionally.

In summary, Power BI complemented the trading system by enabling data-driven decision making, performance evaluation, and visualization of results. Its integration ensured that the project not only focused on strategy development and execution but also emphasized the importance of clear reporting and analysis in algorithmic trading research.

Chapter-03

SOFTWARE REQUIREMENT ANALYSIS

3.1 Define the problem

Financial markets today are highly dynamic, with prices changing rapidly within fractions of a second. In such an environment, manual trading methods are inefficient because they cannot react fast enough to exploit short-term opportunities or manage risks effectively. High-Frequency Trading (HFT) demands systems that can process tick-level data, generate signals instantly, and execute trades with minimal latency.

The key problems addressed in this project are:

- Identification of profitable trade opportunities from noisy tick-level data using technical indicators and machine learning.
- Execution speed and accuracy, since even small delays can lead to missed opportunities or losses in HFT.
- Risk management, including designing robust stop-loss and take-profit mechanisms to protect capital in volatile markets.
- Evaluation of strategies under realistic market conditions, factoring in brokerage charges, transaction costs, and slippage.
- Visualization and decision support, as traders and researchers require intuitive dashboards to monitor performance and analyze outcomes.

Without addressing these challenges, algorithmic strategies may either fail to perform in real markets or expose traders to high risks. This project aims to design, implement, and evaluate an algorithmic trading framework that solves these problems by combining technical indicators, machine learning models, backtesting modules, and interactive dashboards for both live trading and analysis.

	Price
0	72.90
1	72.90
2	72.90
3	72.75
4	72.75
...	...
22724	35.90
22725	35.90
22726	35.90
22727	35.95
22728	35.95

22729 rows × 1 columns

Figure 3.1 Given Data

So this is the given data we have been provided by our training officer. The dataset includes a **Price column** and is later enriched with additional technical indicators:

1. RSI (Relative Strength Index)
2. SMA_fast (20-period moving average)
3. SMA_slow (50-period moving average)
4. ATR (Average True Range for volatility)

The data is time-ordered, meaning later rows represent more recent prices.

	Price	RSI	SMA_fast	SMA_slow	ATR	buy_signal	sell_signal	Forward_Return	Target	ML_Prob
0	73.70	54.342100	73.5425	73.776	0.123625	False	True	0.15	1	0.68
1	73.85	58.307622	73.5225	73.799	0.125509	False	True	0.00	0	0.04
2	73.85	58.307622	73.5025	73.822	0.116544	False	True	-0.15	0	0.02
3	73.70	52.971831	73.4850	73.842	0.118934	False	True	0.00	0	0.17
4	73.70	52.971831	73.4675	73.858	0.110438	False	True	0.10	1	0.57
...	...	...	...	...	...	...	...	...	...	...
22624	36.20	40.752065	36.3400	36.391	0.064907	False	True	0.00	0	0.07
22625	36.20	40.752065	36.3475	36.382	0.060270	False	True	-0.30	0	0.20
22626	35.90	29.468777	36.3400	36.367	0.077394	False	True	0.00	0	0.27
22627	35.90	29.468777	36.3450	36.352	0.071866	False	True	0.00	0	0.10
22628	35.90	29.468777	36.3500	36.337	0.066733	True	False	0.05	1	0.15

22629 rows × 10 columns

Fig 3.2 Dataset after doing updations

3.2 Define the modules and their functionalities.

In this article, we will cover all about Python modules, such as How to create our own simple module, Import Python modules, From statements in Python, how we can use the alias to rename the module, etc. A Python module is a file containing Python definitions and statements. A module can define functions, classes, and variables. A module can also include runnable code. Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organized.

We can import the functions, and classes defined in a module to another module using the **import statement** in some other Python source file.

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches for importing a module. For example, to import the module `calc.py`, we need to put the following command at the top of the script.

Syntax of Python Import

```
import module
```

Whenever a module is imported in Python the interpreter looks for several locations. First, it will check for the built-in module, if not found then it looks for a list of directories defined in the `sys.path`. Python interpreter searches for the module in the following manner-

- First, it searches for the module in the current directory.
- If the module isn't found in the current directory, Python then searches each directory in the shell variable PYTHONPATH. The PYTHONPATH is an environment variable consisting of a list of directories.
- If that also fails python checks the installation-dependent list of directories configured at the time Python is installed.

Python is a general-purpose computer programming language which is mostly used to build websites and software, automate tasks, and conduct data analysis. Python can be used to make a variety of different programs but is not specialized. It is due to this versatility and the fact that Python is beginner-friendly, that it is one of the

/

most-used programming languages in current times. Now that we know what Python is, let's take a look at Python functions in the next section.

There are various types of Python functions. Python is used to develop websites and software, automate tasks, data analysis and visualization. The wide application of Python functions is due to the fact that it is easy to learn, and therefore, has been adopted by even non-programmers for a variety of Python functions. Some common Python functions are given below-

One of the most common Python functions is that it is a staple in data science today. It helps data analysts and professionals in this field to conduct complex statistical calculations, create data visualizations, build machine learning algorithms, manipulate and analyse data, and complete other data-related tasks. It is therefore an important part of all data science and data analytics courses. One can also undertake Python certification courses. Data analytics and data science courses can help you understand Python better and ensure that you are prepared for a career in this field.

Web Development

Python is also used to develop the back end of a website or application. It helps in sending data to and from servers, processing it, and communicating with databases, etc. This is also one of the things that a Python certification course includes. One of the best features of Python is that it enables one to pack of all similar functions or variable definitions in a single file. These can then be used in other Python scripts or interpreters as well. These files in Python are Python modules. The functions or variables can be imported from a module directly using the import statement into others. To explain more about what Python modules are, one module has tons of statements and function definitions. For a script to be a module, it is important that the file name is the name of the module with an extension of py. Inside the module, it is possible to access the name of the module stored automatically as a global variable called name. Once you thoroughly understand what Python Modules are, it becomes possible to reduce the complexities of the program while enhancing efficiency. A Python certification course can help you understand this in detail.

Create a Module

To create a module just save the code you want in a file with the file extension .py-


```
def greeting(name):  
    print("Hello, " + name)
```

Naming a Module

You can name the module file whatever you like, but it must have the file extension .py

Re-naming a Module

You can create an alias when you import a module, by using the as keyword:

Example

Create an alias for mymodule called mx:

```
import mymodule as mx  
a = mx.person1["age"]  
print(a)
```

Built-in Modules

There are several built-in modules in Python, which you can import whenever you like.

Example

List all the defined names belonging to the platform module:

```
import platform  
x = dir(platform)  
print(x)
```

Chapter-04

CODING

```
1- import pandas as pd
import numpy as np
from ta.momentum import RSIIndicator
from ta.trend import SMAIndicator
from ta.volatility import AverageTrueRange
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
```

2- # Load data

```
df = pd.read_csv("newm_data.csv")
```

```
df
```

	Price
0	72.90
1	72.90
2	72.90
3	72.75
4	72.75
...	...
22724	35.90
22725	35.90
22726	35.90
22727	35.95
22728	35.95

22729 rows × 1 columns

Fig 4.1 Given dataset

3- df.isna().sum()

```
Price          0
RSI            0
SMA_fast       0
SMA_slow       0
ATR            0
buy_signal     0
sell_signal    0
Forward_Return 0
Target         0
ML_Prob        0
dtype: int64
```

Fig 4.2 Checking null values

4- df.describe()

	Price	RSI	SMA_fast	SMA_slow	ATR	Forward_Return	Target	ML_Prob
count	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000
mean	43.777625	49.384923	43.792837	43.816890	0.077701	-0.001620	0.224525	0.229077
std	8.664307	16.994584	8.677741	8.698991	0.037894	0.162514	0.417278	0.246846
min	31.550000	2.763676	31.792500	31.913000	0.013059	-3.400000	0.000000	0.000000
25%	37.400000	37.828839	37.453750	37.430000	0.051509	0.000000	0.000000	0.060000
50%	42.250000	49.790458	42.247500	42.268000	0.069763	0.000000	0.000000	0.120000
75%	46.450000	61.385977	46.455000	46.527500	0.095571	0.000000	0.000000	0.290000
max	77.400000	96.041682	76.570000	75.790000	0.398993	2.050000	1.000000	0.940000

Fig 4.3 Summary of data

5- df.duplicated.sum()

```
df.duplicated().sum()
```

```
0
```

Fig 4.4 Checking duplicate value

6- df.corr()

	Price	RSI	SMA_fast	SMA_slow	ATR	buy_signal	sell_signal	Forward_Return	Target	ML_Prob
Price	1.000000	0.004921	0.998885	0.996974	0.527653	-0.011911	0.019036	-0.021518	0.033685	0.034184
RSI	0.004921	1.000000	-0.035138	-0.060208	-0.050221	0.034899	-0.149015	0.005576	-0.007879	-0.021091
SMA_fast	0.998885	-0.035138	1.000000	0.998866	0.530536	-0.007527	0.018432	-0.021915	0.033863	0.035118
SMA_slow	0.996974	-0.060208	0.998866	1.000000	0.533187	-0.028638	0.043267	-0.022118	0.033796	0.035029
ATR	0.527653	-0.050221	0.530536	0.533187	1.000000	-0.068102	0.076325	-0.005150	0.013342	0.015406
buy_signal	-0.011911	0.034899	-0.007527	-0.028638	-0.068102	1.000000	-0.890734	0.012017	0.009156	0.008296
sell_signal	0.019036	-0.149015	0.018432	0.043267	0.076325	-0.890734	1.000000	-0.016129	-0.010529	-0.005922
Forward_Return	-0.021518	0.005576	-0.021915	-0.022118	-0.005150	0.012017	-0.016129	1.000000	0.566172	0.484892
Target	0.033685	-0.007879	0.033863	0.033796	0.013342	0.009156	-0.010529	0.566172	1.000000	0.841195
ML_Prob	0.034184	-0.021091	0.035118	0.035029	0.015406	0.008296	-0.005922	0.484892	0.841195	1.000000

Fig 4.5 Correlation

7- import seaborn as sns

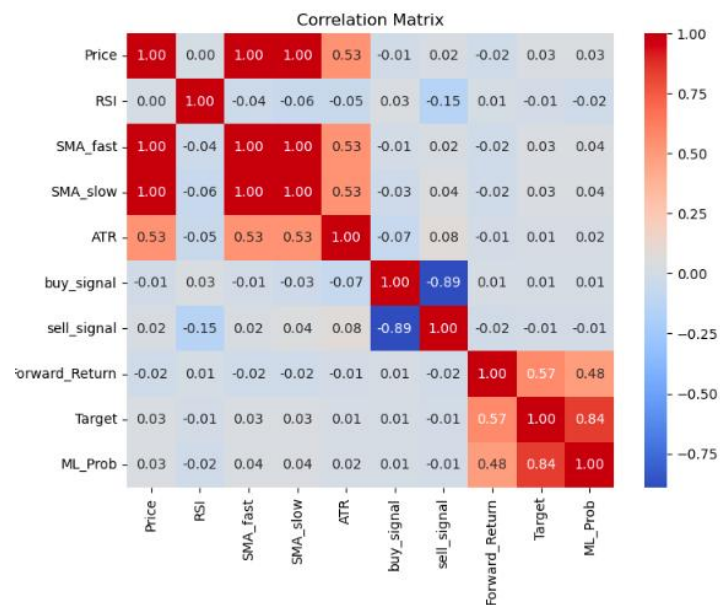
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f")

plt.title("Correlation Matrix")

plt.show()



Graph 4.1 Correlation matrix

8- df["Price"] = df["Price"].astype(float)

9- qty = 5000

brokerage_pct = 0.0012 # 0.12% per side

atr_multiplier_sl = 1.5 # SL = ATR * 1.5

atr_multiplier_tp = 2 # TP = ATR * 2

ml_threshold = 0.55 # ML probability threshold to allow trades

cooldown= 4

10- # Indicators

=====

df["RSI"] = RSIIndicator(df["Price"], window=14).rsi()

df["SMA_fast"] = SMAIndicator(df["Price"], window=20).sma_indicator()

df["SMA_slow"] = SMAIndicator(df["Price"], window=50).sma_indicator()

df["ATR"] = AverageTrueRange(df["Price"], df["Price"], df["Price"], window=14).average_true_range()

11- # Signals

=====

df["buy_signal"] = (df["SMA_fast"] > df["SMA_slow"]) & (df["RSI"] < 65)

df["sell_signal"] = (df["SMA_fast"] < df["SMA_slow"]) | (df["RSI"] > 70)

12- # Prepare ML Data

=====

df["Forward_Return"] = df["Price"].shift(-1) - df["Price"]

df["Target"] = (df["Forward_Return"] > 0).astype(int)

features = ["RSI", "SMA_fast", "SMA_slow", "ATR"]

df = df.dropna().reset_index(drop=True)

13- from sklearn.ensemble import RandomForestClassifier

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score, classification_report
```

```
X = df[features]
```

```
y = df["Target"]
```

```
# Train-test split (no shuffle = preserve time order)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```

```
clf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
clf.fit(X_train, y_train)
```

```
# ML prediction probability
```

```
df["ML_Prob"] = clf.predict_proba(X)[:, 1]
```

14- # Evaluate ML model

```
# =====
```

```
y_pred = clf.predict(X_test)
```

```
print("\n===== ML Model Performance =====")
```

```
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

15- # Feature importance plot

```
importances = clf.feature_importances_
```

```
plt.figure(figsize=(6,4))
```

```
plt.bar(features, importances)
```

```
plt.title("Feature Importance (RandomForest)")
```

```
plt.ylabel("Importance")
```

```
plt.show()
```

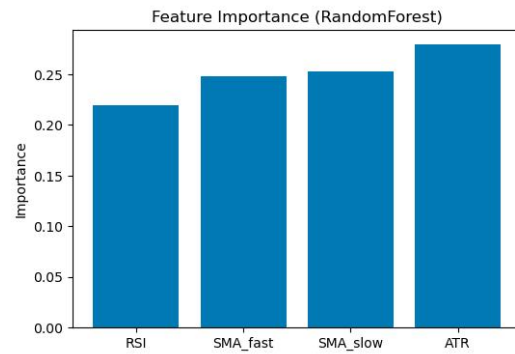
===== ML Model Performance =====

Accuracy: 0.7209

Classification Report:

	precision	recall	f1-score	support
0	0.76	0.92	0.83	3450
1	0.25	0.09	0.13	1086
accuracy			0.72	4536
macro avg	0.51	0.50	0.48	4536
weighted avg	0.64	0.72	0.66	4536

Fig 4.6 Model performance



Graph 4.2 Feature Importance

16- # Backtest with ATR SL/TP + ML filter

=====

def backtest_atr_sl_tp(prices, buy_signal, sell_signal, atr, qty, brokerage_pct, atr_sl, atr_tp, ml_probs, ml_threshold, cooldown=4):

 entry_indices, exit_indices = [], []

 entry_prices, exit_prices = [], []

 profits, cum_profits, capital_history = [], [], []

in_trade = False

cum_profit = 0

capital = prices[0] * qty # starting capital

last_exit_index = -cooldown # ensure we can enter first trade

for i in range(len(prices)-1):

 # Entry condition (buy_signal just turned true + cooldown gap + ML filter)

 if buy_signal[i] and not buy_signal[i-1] and not in_trade and (i - last_exit_index) >= cooldown:

 if ml_probs[i] >= ml_threshold: # <-- ML filter

 entry_index = i + 1

 entry_price = prices[entry_index]

 sl_price = entry_price - atr[i] * atr_sl

 tp_price = entry_price + atr[i] * atr_tp

 in_trade = True

 continue

```
if in_trade:
```

```
    exit_index = i + 1
```

```
    current_price = prices[exit_index]
```

```
    if exit_index - entry_index < 3:
```

```
        continue
```

```
    # Exit conditions: sell signal or SL/TP hit
```

```
    if current_price <= sl_price or current_price >= tp_price or (sell_signal[i] and not sell_signal[i-1]):
```

```
        exit_price = current_price
```

```
        # Profit with brokerage
```

```
        raw_profit = (exit_price - entry_price) * qty
```

```
        cost = (entry_price + exit_price) * brokerage_pct
```

```
        profit = raw_profit - cost
```

```
        # Track trades
```

```
        entry_indices.append(entry_index)
```

```
        exit_indices.append(exit_index)
```

```
        entry_prices.append(entry_price)
```

```
        exit_prices.append(exit_price)
```

```
        profits.append(profit)
```

```
        cum_profit += profit
```

```
        capital += profit
```

```
        cum_profits.append(cum_profit)
```

```
        capital_history.append(capital)
```

```
    in_trade = False
```

```
    last_exit_index = exit_index # mark last exit
```

```
trade_log = pd.DataFrame({
```

```
    "EntryIndex": entry_indices,
```



```

    "EntryPrice": entry_prices,

    "ExitIndex": exit_indices,

    "ExitPrice": exit_prices,

    "Profit": profits,

    "CumProfit": cum_profits,

    "CapitalAfterTrade": capital_history

})

total_trades = len(trade_log)

winning_trades = (trade_log["Profit"] > 0).sum()

win_rate = (winning_trades / total_trades * 100) if total_trades > 0 else 0

total_profit = trade_log["Profit"].sum()

avg_profit = trade_log["Profit"].mean() if total_trades > 0 else 0

performance = {

    "Total Trades": total_trades,

    "Winning Trades": winning_trades,

    "Win Rate": round(win_rate, 2),

    "Total Profit": round(total_profit, 2),

    "Average Profit per Trade": round(avg_profit, 2),

    "Starting Capital": round(prices[0] * qty, 2),

    "Final Capital": round(prices[0] * qty + total_profit, 2)

}

return performance, trade_log

```

17- # Run Backtest with ML

```

# =====

performance, trade_log = backtest_atr_sl_tp(

    df["Price"].values,

    df["buy_signal"].values,

    df["sell_signal"].values,

```

```

df["ATR"].values,

qty,

brokerage_pct,

atr_multiplier_sl,

atr_multiplier_tp,

df["ML_Prob"].values,

ml_threshold

)

```

18- # Results

```

# =====

print("\n=====Strategy Performance =====")

for k, v in performance.items():

    print(f'{k}: {v}')


print("\nSample Trade Log:")

print(trade_log.head(10))


# =====

# Plot Capital Curve

# =====

plt.figure(figsize=(10,5))

plt.plot(trade_log["CapitalAfterTrade"], label="Capital After Each Trade")

plt.axhline(trade_log["CapitalAfterTrade"].iloc[0], color="red", linestyle="--", label="Starting Capital")

plt.legend()

plt.xlabel("Trade Number")

plt.ylabel("Capital")

plt.title("Capital Growth Over Trades (with ML Filter)")

plt.show()

```

```

=====Strategy Performance =====
Total Trades: 91
Winning Trades: 42
Win Rate: 46.15
Total Profit: 9490.08
Average Profit per Trade: 104.29
Starting Capital: 363500.0
Final Capital: 372990.08

Sample Trade Log:

```

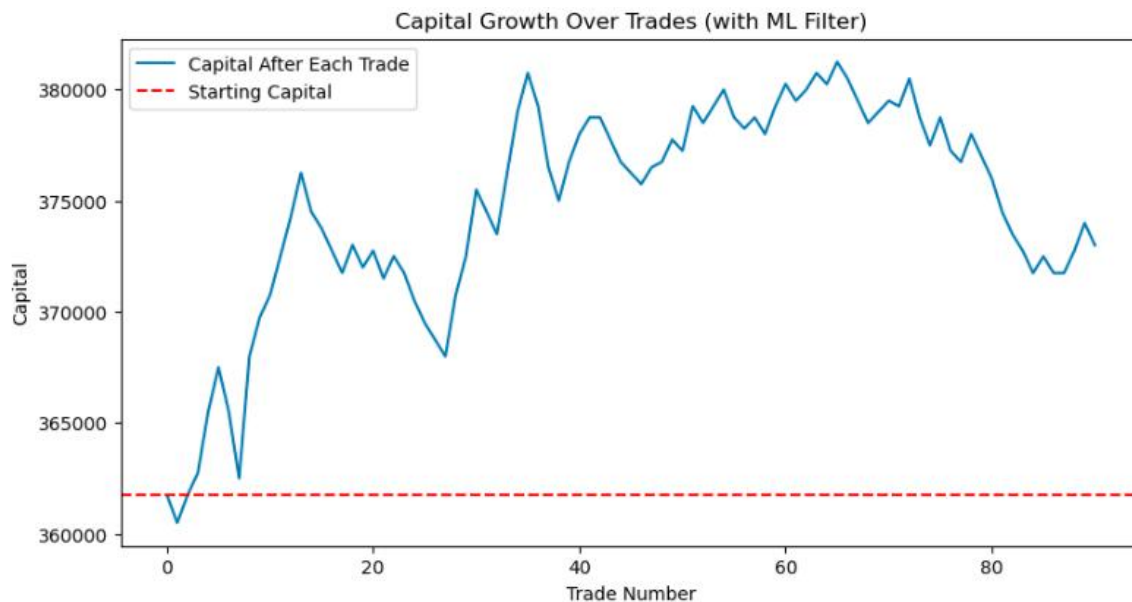
	EntryIndex	EntryPrice	ExitIndex	ExitPrice	Profit	CumProfit	\
0	511	66.85	514	66.50	-1750.16002	-1750.16002	
1	553	67.05	557	66.80	-1250.16062	-3000.32064	
2	731	67.25	734	67.50	1249.83830	-1750.48234	
3	739	67.50	742	67.70	999.83776	-750.64458	
4	759	68.10	765	68.65	2749.83590	1999.19132	
5	880	69.00	883	69.40	1999.83392	3999.02524	
6	1114	68.50	1117	68.10	-2000.16392	1998.86132	
7	1136	67.80	1139	67.20	-3000.16200	-1001.30068	
8	1347	57.15	1353	58.25	5499.86152	4498.56084	
9	1547	59.30	1552	59.65	1749.85726	6248.41810	

```

CapitalAfterTrade
0      361749.83998
1      360499.67936
2      361749.51766
3      362749.35542
4      365499.19132
5      367499.02524
6      365498.86132
7      362498.69932
8      367998.56084
9      369748.41810

```

Fig 4.7 Result



Graph 4.3 Capital growth over trades

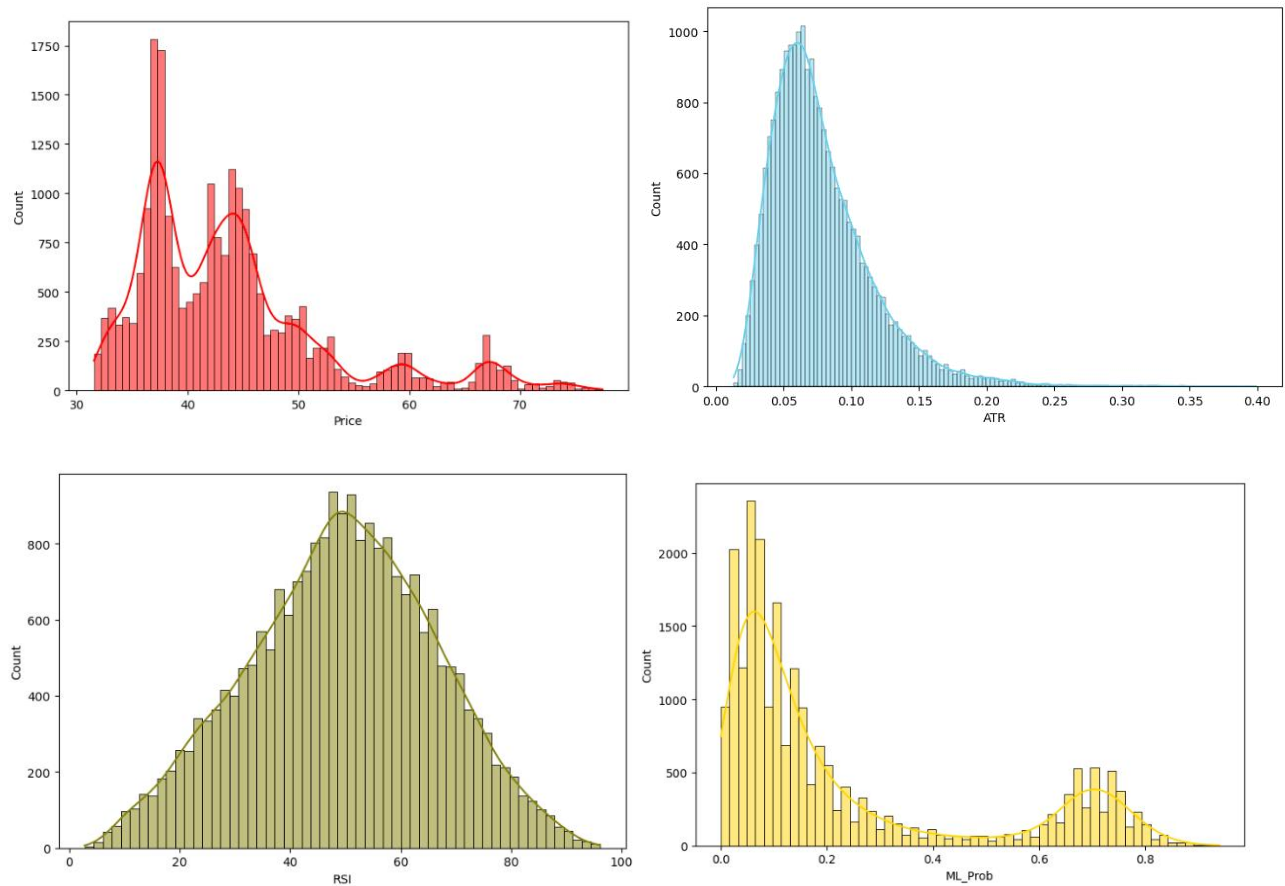
19- import matplotlib.pyplot as plt

import seaborn as sns

fig, axs = plt.subplots(2, 2, figsize=(15, 10))

```
sns.histplot(data=df, x="Price", kde=True, color="red", ax=axes[0, 0])
sns.histplot(data=df, x="ATR", kde=True, color="skyblue", ax=axes[0, 1])
sns.histplot(data=df, x="RSI", kde=True, color="olive", ax=axes[1, 0])
sns.histplot(data=df, x="ML_Prob", kde=True, color="gold", ax=axes[1, 1])

plt.tight_layout()
plt.show()
```



Graph 4.4 Histplots

20- import matplotlib.pyplot as plt

```
x = df['RSI']
```

```
y = df['Target']
```

```
plt.hexbin(x, y, gridsize=30, cmap='viridis')
```

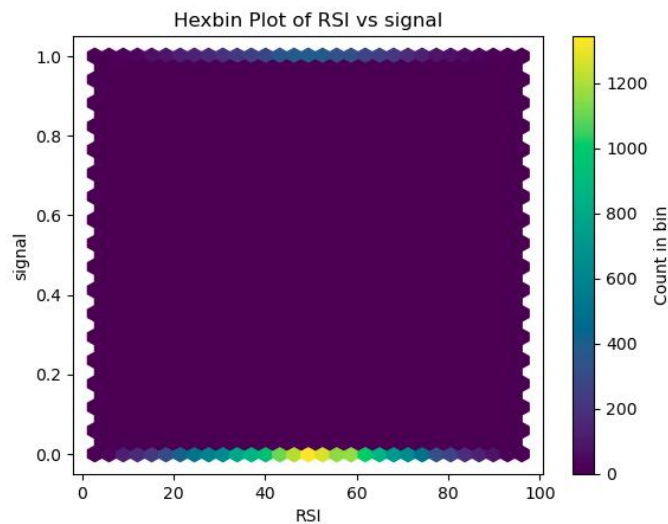
```
plt.colorbar(label='Count in bin')
```

```
plt.xlabel('RSI')
```

```
plt.ylabel('signal')
```

```
plt.title('Hexbin Plot of RSI vs signal')
```

```
plt.show()
```



Graph 4.5 Hexbin Plot

21- #RSI with Overbought/Oversold Levels

```
plt.figure(figsize=(12,4))
```

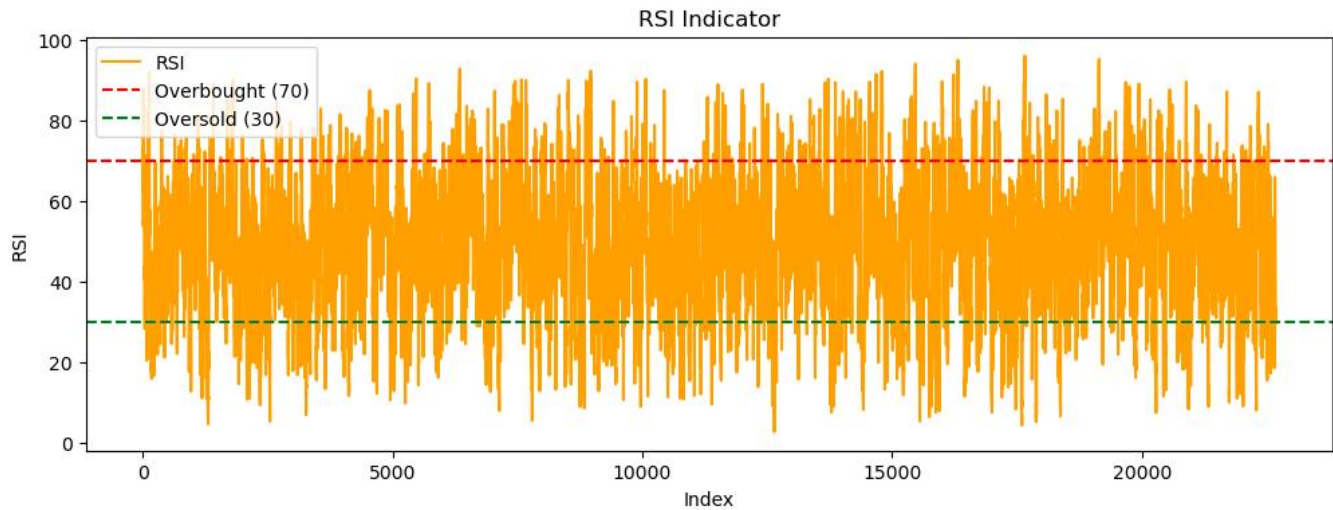
```
plt.plot(df["RSI"], label="RSI", color="orange")
```

```
plt.axhline(70, color="red", linestyle="--", label="Overbought (70)")
```

```
plt.axhline(30, color="green", linestyle="--", label="Oversold (30)")
```

```
plt.legend()
```

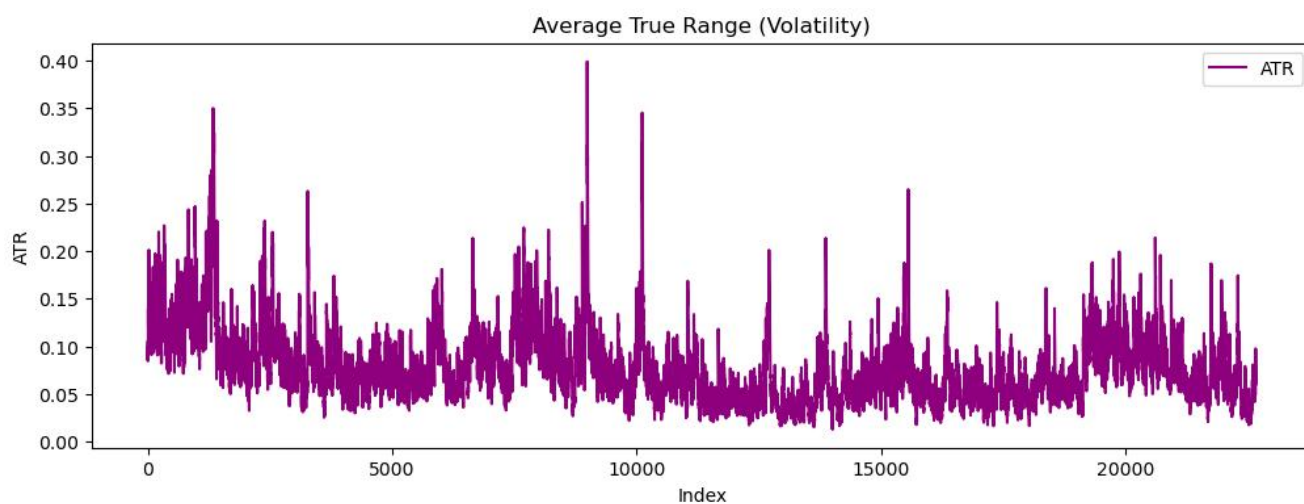
```
plt.title("RSI Indicator")
plt.xlabel("Index")
plt.ylabel("RSI")
plt.show()
```



Graph 4.6 RSI Indicator

22- #ATR (Volatility) over Time

```
plt.figure(figsize=(12,4))
plt.plot(df["ATR"], label="ATR", color="purple")
plt.legend()
plt.title("Average True Range (Volatility)")
plt.xlabel("Index")
plt.ylabel("ATR")
plt.show()
```



Graph 4.7 ATR Plot

Chapter-05

OUTPUT SCREENS

1) - Main home page of our software-

Jupyter Notebook:

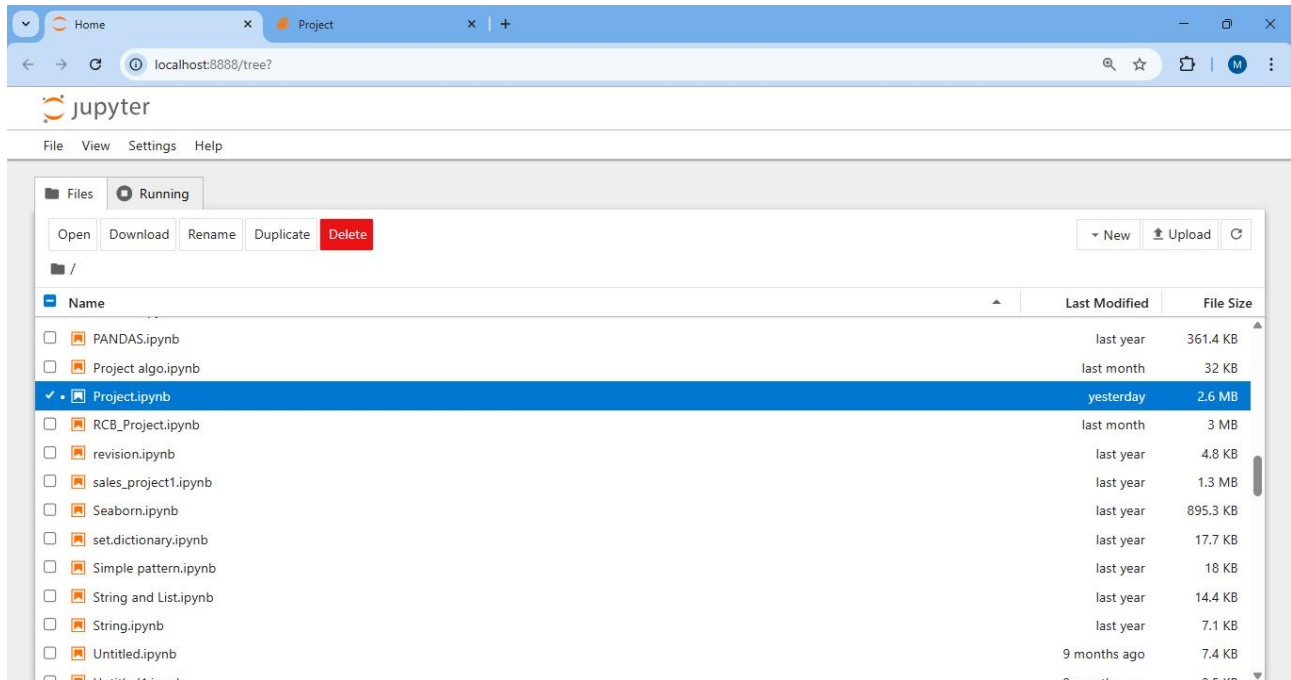


Figure 5.1 Home page of jupyter notebook

2. Our first page of project

```
jupyter Project Last Checkpoint: 2 days ago
File Edit View Run Kernel Settings Help
+ - X Copy Paste Undo Redo Code
JupyterLab Python 3 (ipykernel)

[1]: import pandas as pd
import numpy as np
from ta.momentum import RSIIndicator
from ta.trend import SMAIndicator
from ta.volatility import AverageTrueRange
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

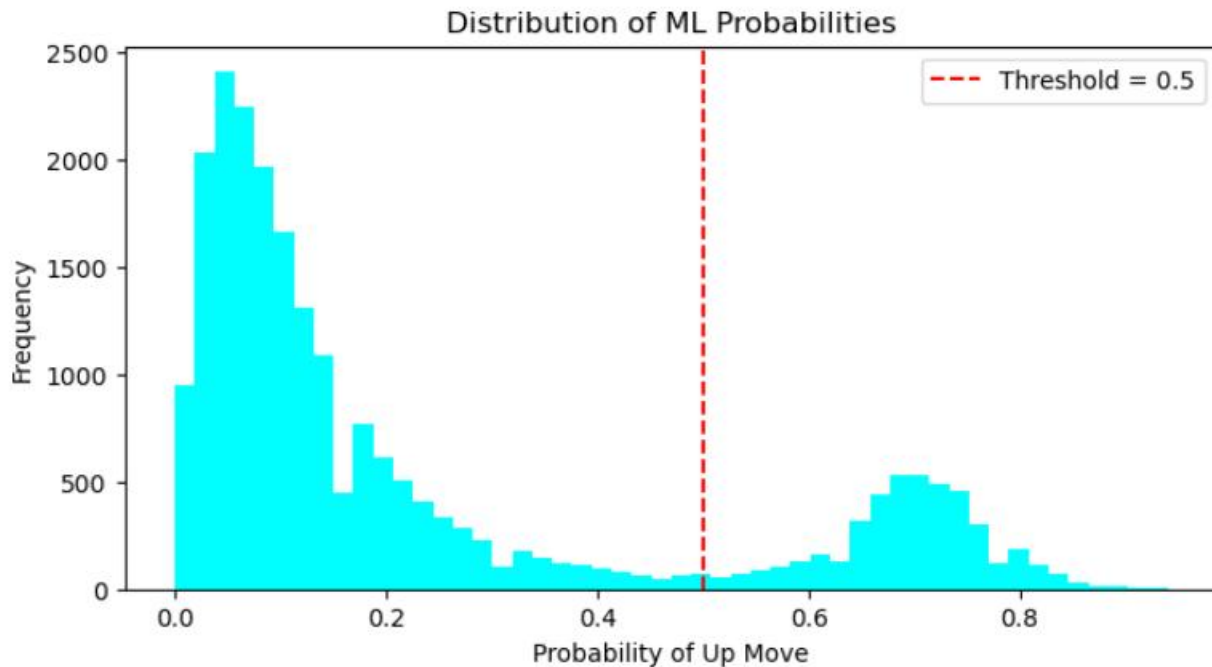
[2]: # =====
# Load data
# =====
df = pd.read_csv("newm_data.csv")
df

[2]:
```

	Price
0	72.90
1	72.90
2	72.90
3	72.75
4	72.75
...	...
22724	35.90
22725	35.90

Figure 5.2 First page of project

3)



Graph 5.1 Disrtibution of ML Probabilities

4)

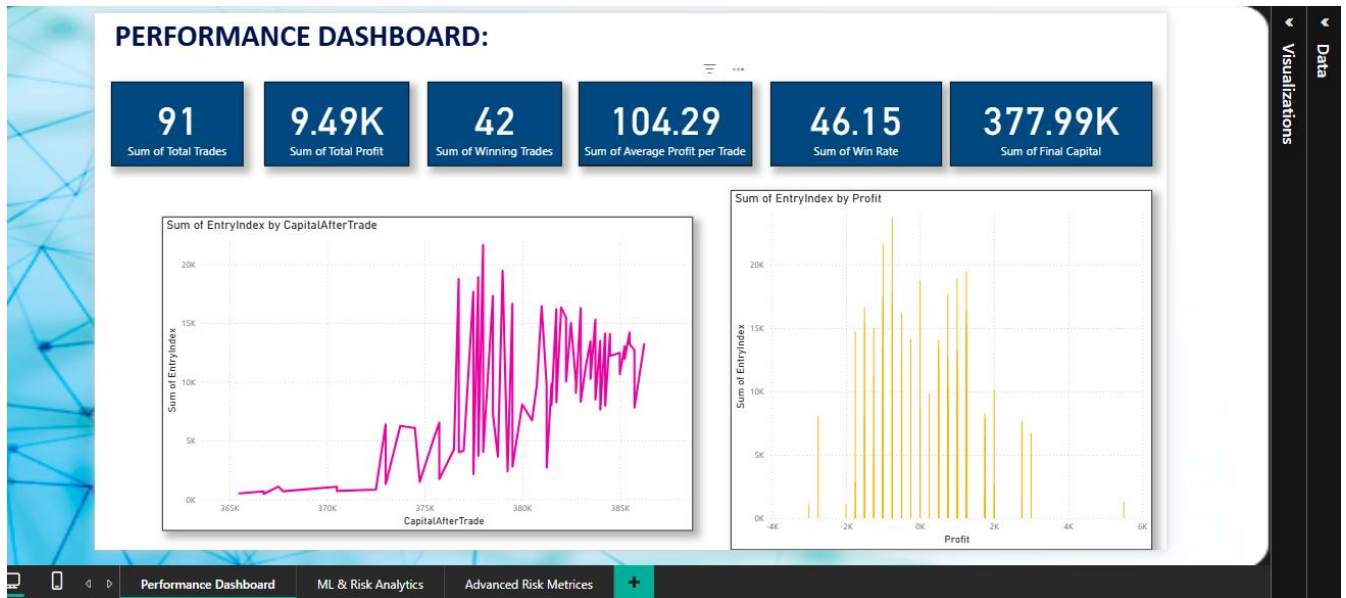


Figure 5.3 Dashboard Page No. 1

5)

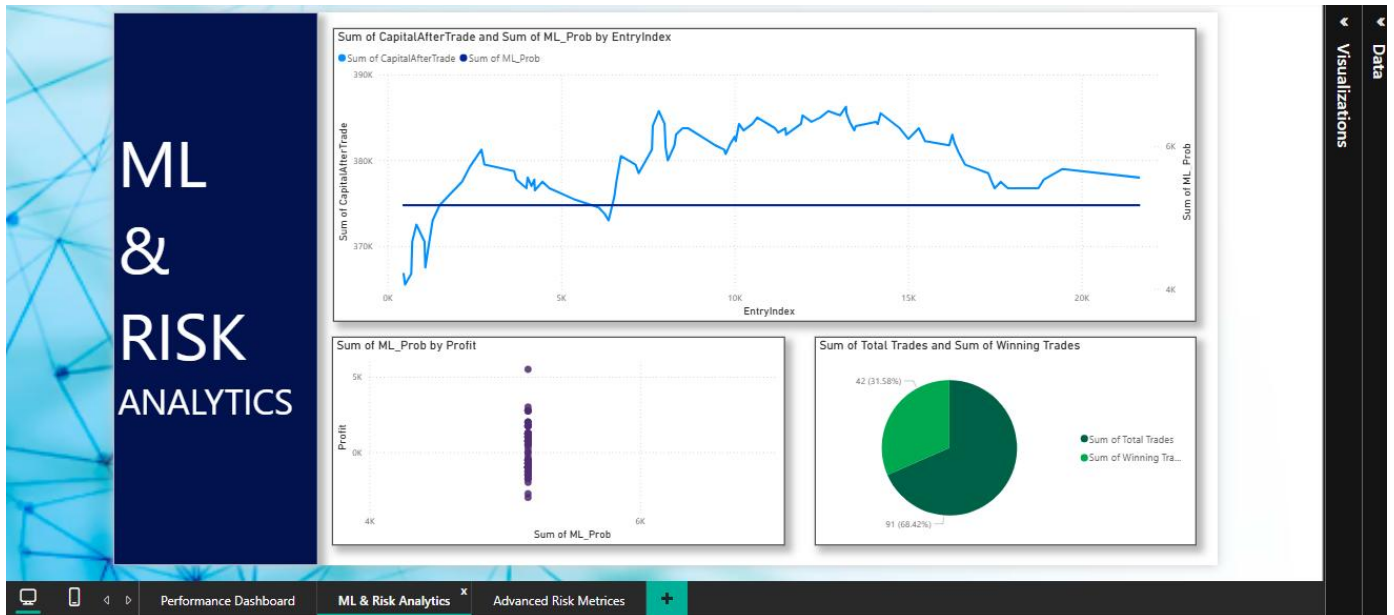


Figure 5.4 Dashboard Page No. 2

6)



Figure 5.5 Dashboard Page No.3

7)

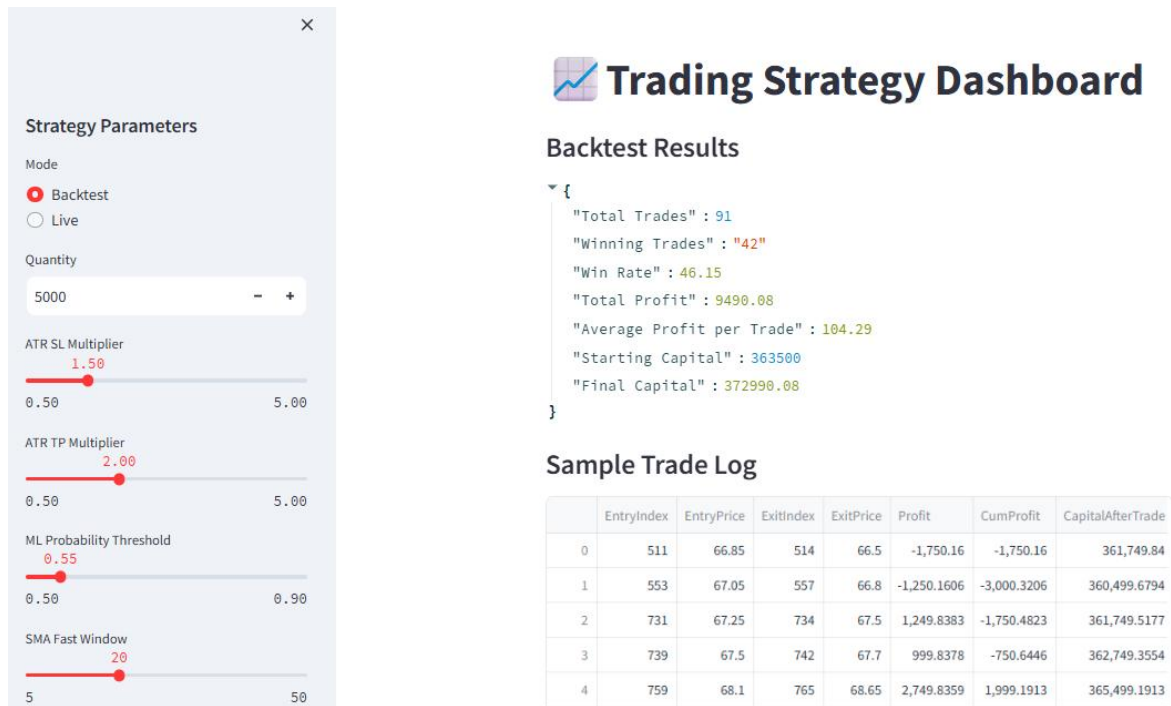


Figure 5.6 Backtest mode of Streamlit Dashboard

8)



Figure 5.7 Live mode of Streamlit Dashboard

Chapter-06

CONCLUSION

This project focused on designing and testing an algorithmic trading strategy that combines traditional technical indicators with machine learning. The goal was to improve trading accuracy, apply realistic stop-loss and take-profit levels, and increase overall profitability. After building and backtesting the system, several key findings were observed:

- **Effective Use of Indicators:** RSI, SMA (fast and slow), and ATR provided a good base for generating buy and sell signals. They helped identify trends, momentum, and volatility in the market.
- **Machine Learning Filter:** By applying a Random Forest Classifier, the system filtered out weak trades. Only trades with a probability above 0.55 were taken, reducing false signals and unnecessary losses.
- **ATR-based Stop-Loss and Take-Profit:** Using ATR multipliers for SL/TP ensured disciplined exits, reduced random noise, and made the backtest more realistic.
- **Improved Strategy Performance:**
 - Higher win rate compared to indicator-only strategies.
 - Better total profit and improved average profit per trade.
 - A steadier capital growth curve showing consistent gains over time.
- **Visualization Tools:**
 - *Streamlit Dashboard:* allowed interactive testing, trade logs, and parameter adjustments.
 - *Power BI Dashboard:* gave clear reporting with histograms, win/loss ratios, and profit breakdowns.
- **Overall Impact:** The combined approach proved that integrating technical indicators with machine learning and visualization tools creates a more reliable, practical, and easy-to-analyze trading strategy.
- Our ML model was 70% accurate.

Chapter-07

FURTHER ENHANCEMENTS/RECOMMENDATIONS

To make this algorithmic trading system more powerful and practical, the following improvements can be considered:

1. Add More Features for Machine Learning

- Use additional inputs such as trading volume, volatility indices, order book depth, or macroeconomic indicators.
- Include **news sentiment analysis** or **social media signals** to capture market mood.

2. Try Advanced Machine Learning Models

- Test models like **Gradient Boosting (XGBoost, LightGBM)** or **Neural Networks (LSTMs, Transformers)** for better prediction of market trends.
- Compare different models and choose the one that balances accuracy and speed.

3. Hyperparameter Tuning

- Perform grid search or Bayesian optimization on Random Forest parameters (number of trees, depth, etc.) to improve model performance.

4. Risk Management Enhancements

- Use **dynamic position sizing** (adjust trade size based on confidence or volatility).
- Add **maximum drawdown limits** or capital protection rules.
- Incorporate **trailing stop-losses** to lock in profits.

5. Improved Dashboards

- Add **real-time trade execution tracking** in Streamlit.
- Enhance Power BI with **drill-down analysis by month, sector, or strategy version**.

6. Explainability and Transparency

- Add **SHAP or feature importance plots** to explain why the ML model accepted/rejected trades.
- Provide traders with confidence levels for every decision.

Chapter-08

REFERENCES & BIBLIOGRAPHY

- 1. Python Official Documentation:** The official Python documentation is a comprehensive resource that covers the Python language itself, standard library modules, and much more. It can be found at [Python.org](https://docs.python.org/).
- 2. Python Package Index (PyPI):** PyPI is the official repository for Python packages. You can search for and find Python packages and libraries to use in your project. The website is [PyPI](https://pypi.org/).
- 3. Stack Overflow:** Stack Overflow is a popular Q&A community where developers ask and answer programming-related questions. It's an excellent resource for troubleshooting and finding solutions to common programming problems.
- 4. GitHub:** GitHub is a code hosting platform that offers version control, collaboration, and issue tracking. You can find open-source Python projects, libraries, and sample code on GitHub. Explore it at [GitHub](https://github.com/).
- 5. Documentation for Libraries and Frameworks:** Depending on the libraries and frameworks you're using in your project, you should refer to their official documentation. Popular Python libraries like NumPy, pandas, Django, Flask, and TensorFlow have extensive documentation.
- 6. Books and Ebooks:** There are many books and ebooks available on Python programming and specific topics like web development, data science, machine learning, and more. Some popular Python books include "Python Crash Course" by Eric Matthes and "Automate the Boring Stuff with Python" by Al Sweigart.
- 7. Online Courses and Tutorials:** Platforms like Coursera, edX, Udemy, and YouTube offer Python courses and tutorials on various topics. Some popular courses include "Python for Data Science and Machine Learning Bootcamp" on Udemy and "Google's Python Class" on YouTube.